Unit 2. Ensemble Modeling

Machine Learning 1 (ML1)

Master in Artificial Intelligence (MIA)

October 27, 2025

2.1 Introduction

In this unit we are going to discuss models composed of **multiple learners** that **complement each other**.

- By combining them, we attain higher accuracy.

Why is that useful?

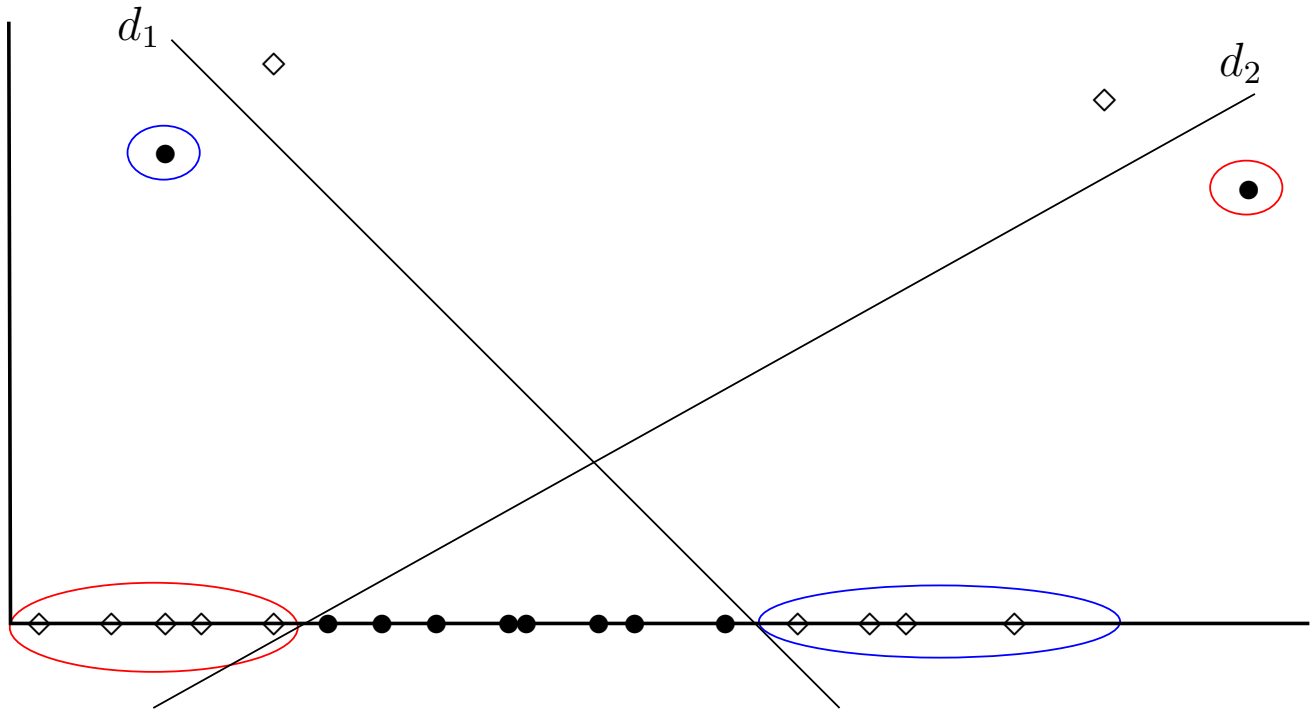
No Free Lunch Theorem (Wolpert & Macready): there is no single learning algorithm that in every domain always induces the most accurate learner.

- **Increased performance** for an algorithm in **one class** of problems is offset by **lower performance** in **other classes**.
- **Usual approach**: try many and choose the one that performs the **best** on a separate **validation set**. But:
  - Each **learning algorithm** is based on a set of **assumptions** that do not necessarily hold for the data.
  - With finite data, each algorithm converges to a **different solution** and fails under different circumstances.
  - **Fine-tuning** to get the biggest possible accuracy on the validation set can be **difficult** and **time consuming**.
    - There may be **instances** in which the learner **still fails**.
    - **Solution**: **find another learner** that is more accurate on those. Or maybe more than one.

Instead of relying in a single higly-tuned highly-accurate learner, we can train simpler **base learners** that **complement** each other, to improve performance.

- The final decision is based in the **combination** of opinions from experts (the trained models).
- We are interested in models with **different decision boundaries**.
- The **combination** of the models gives us a decision boundary **better suited** to the problem.

E.g., we have a binary classification problem (classes $\bullet$ and $\diamond$), and two linear classifiers d_1 and d_2, none of which is very good.



- d_1 classifies as $\bullet$ the instances left of its decision boundary, and as $\diamond$ the rest $\Rightarrow$ it misclassifies samples in red.
- d_2 classifies as $\bullet$ the instances under its decision boundary, and as $\diamond$ the rest $\Rightarrow$ it misclassifies samples in blue.
- **Misclassified samples** by d_1 are **different** that those by d_2.
  - The **outputs** of the classifiers are **complementary**.
  - It can be useful to **combine** them.
  - Possible combination:
    1. If d_1 and d_2 predict $\bullet$ $\Rightarrow$ ensemble predicts $\bullet$
    2. Otherwise $\Rightarrow$ ensemble predicts $\diamond$

When designing ensembles, we have to answer two **fundamental questions**:

1. How to generate **base learners** that **complement** each other?
2. How to **combine the outputs** of base learners for **maximum accuracy**?

And we have to answer them in a way that allows us to **maximize the diversity** of base learners more than in individual accuracy:

- For ensembles to work better than a single learner, we need **complementary learners** that do not produce exactly the same outputs.
- **Diverse learners** are accurate in different instances $\Rightarrow$ in other words, they make **different mistakes**.
  E.g., if d_L misclassifies x, but $d_1, d_2, \ldots, d_{L-1}$ (or a mayority of them) do not, a mayority vote would suffice to assure a correct prediction.

On the other hand, we want base learners that are **reasonably accurate**:

- **Not** necessarily **optimized** (separatedly) for the **best accuracy**.
  For some ensembles to work, error rate just has to be lower than the one obtained from random guessing.
- **Simplicity** (lack of complexity) is important.
- We want to strike a **balance** between:
  - Minimizing the **computational costs** of the system and of training.
  - Getting **reasonably good** base learners.

Diversity of base learners is affected by **multiple factors**, that can lead to **different design choices**. Let's examine some of them.

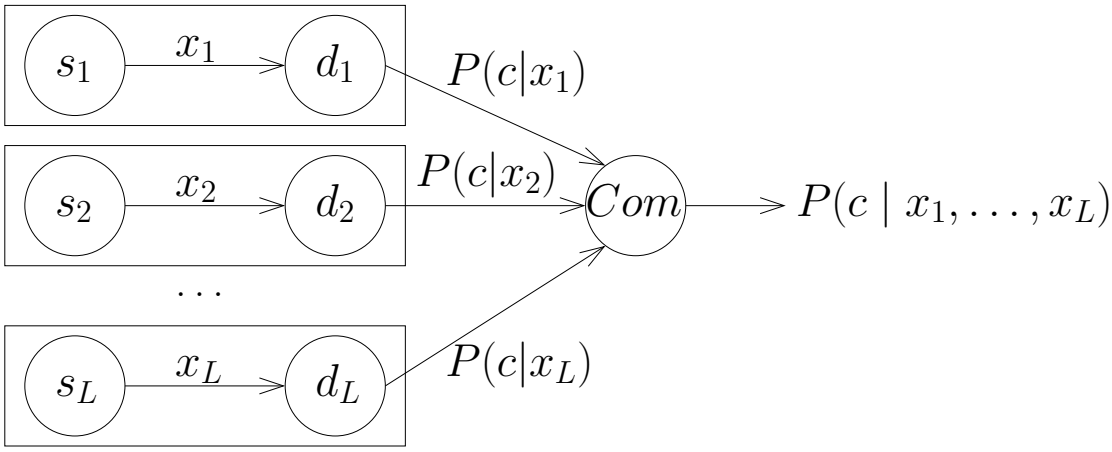
Learning algorithms

Two fundamental choices:

1. Use the **same learning algorithm** for all the base learners.
   - More **simplicity**.
   - Results are **easier to interpret**.
   - **Necessary** in some ensemble algorithms (e.g., random forests).
2. Use **different learning** algorithms.
   - **Different algorithms** make **different assumptions** about the data, leading to different classifiers $\Rightarrow$ **more diversity**.
     E.g., we can combine parametric and non parametric base learners.

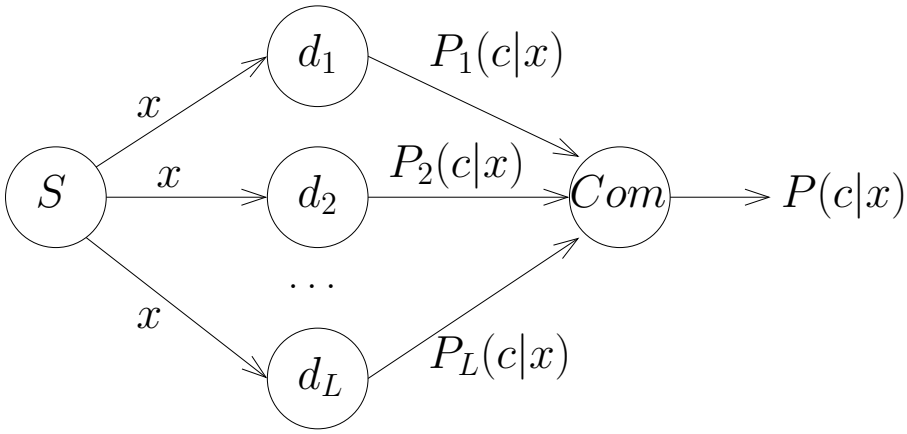
Training data

1. We can **partition** the input data in the **feature space** (**vertical partition**):
   - Each base learner has a **different representation** (different features) of the same dataset.



     - There are L **feature spaces**, $s_1 \ldots s_L$, so each **instance** x has L **representations** $x_1, \ldots x_L$, and L **classifiers** (base learners) $d_1 \ldots d_L$.
     - Their outputs (posterior probabilities) are fed to a **combination rule** or **combiner** Com, responsible for the final prediction.
     - Com can be another **classifier**, combining its input probabilities to estimate $P(c \mid x_1, \ldots, x_L)$
   - **Different representations** make different characteristics explicit $\Rightarrow$ allows **better prediction**.
   - Useful approach when there are **multiple sources** of information.
     - E.g., **Integration** of different types of **sensors/measurements**.
   - Usually **classifiers** of the **same type**.
   - We have to determine the **best** learners for the **basic classifiers** and the **best combination rule**.
2. We can **partition** the training dataset in **different subsets** (**horizontal partition**), or draw **random samples** of the original dataset for training (e.g., bagging).

   Base learners trained in **different instances** $\Rightarrow$ **more diversity**.



3. We can train our base learners with **different target classes** $\Rightarrow$ each individual classifier solves a different classification task.

   E.g., a multiclass problem in which base learners do one-vs-one (binary) classification.

4. We can also **train** base learners **serially**:
   - each learner focus on instances on which previous learners were not confident (e.g., cascading).
   - instances are weighted according to the difficulty in their classification by previous learners (e.g., boosting).

Serial or parallel architectures

Another design choice is how to combine the outputs of the base learners:

1. **Multiexpert combination**: the outputs of the base learners, working in **parallel**, are the inputs of the combination rule (e.g., voting, stacking).
2. **Multistage combination**: ensemble relies on a **sequence of classifiers**.
   - The next base learner is trained with and tested on the instances on which previous base learners were not confident enough (e.g., cascading), or more weight is given to instances on which previous learners failed (e.g., boosting).
   - All base learners are used on the prediction (e.g., boosting), or only the simplest one capable of give an answer with enough confidence (e.g., cascading).

3. **Hierarchical**: the **base learners** are combined **hierarchically**, as nodes of trees or directed graphs, so the output of each learner is directed to the learner of its parent node, or the one at the end of a directed edge.

The kind of **output combination** has an impact on **diversity**:

- Multiexpert combination: each **base learner** is consulted for all inputs.
  - It should be **accurate everywhere**.
  - **Diversity** has to be enforced **everywhere** in the input space.
- Multistage combination: the **input space** is partitioned in **regions of expertise** for different learners.
  - **Diversity** is guaranteed by the **partitioning** of the data.
  - Base learners need to be **accurate** only in their own **local domains**.

Hyperparameters

When using a **single learning algorithm**, **different learners** can be generated by changing their **hyperparameters**.

- Hidden units, initial weights, dropout, ... in ANNs.
- Value of k in k-NNs.
- Kernel function and its parameters in SVMs.

By training multiple base learners with different hyperparameter values, we can **average** over this factor, reducing **error**.

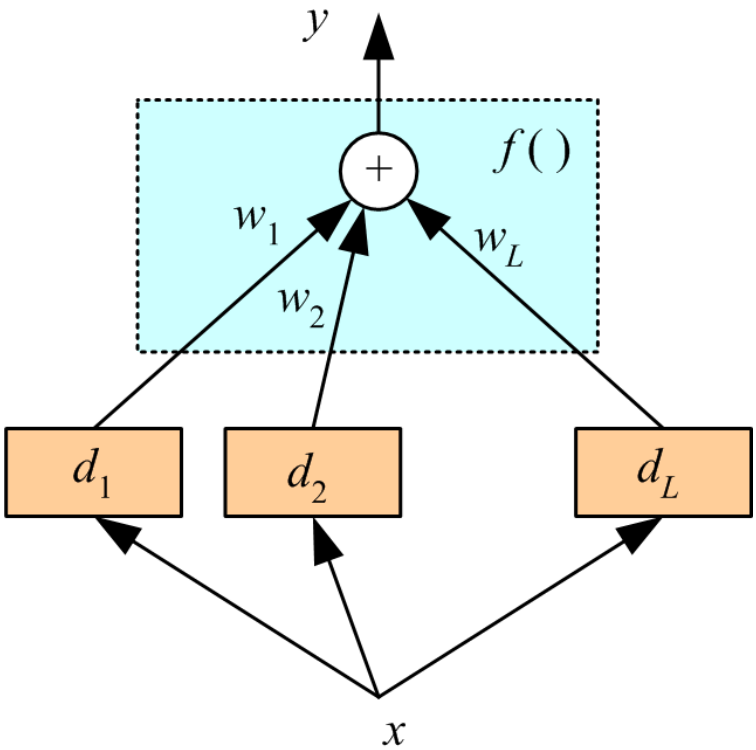
Now let focus our attention on the different methods for ensemble combination:

1. Voting.
2. Combining continuously-valued outputs.
3. Stacking.
4. Cascading.
5. Bagging.
6. Randomization (Random Forests).
7. Boosting.

2.2 Voting

The **simplest way** to combine multiple classifiers is by **voting**, which corresponds to taking a **linear combination** of the learners.

This is how it looks in general, given L **base classifiers** $\boldsymbol{d_1, \ldots, d_L}$, and **weights** $\boldsymbol{w_i}$, $1 < i \leq L$.



Source: "Introduction to Machine Learning", Ethem Alpaydin, Ch 17

More formally:

- Given L **base classifiers**, K **classes**, and an **instance** x.

$$d_{ij}(x) = \begin{cases} 1 & \text{if classifier } d_i \text{ outputs label } j \text{ for } x \\ 0 & \text{otherwise} \end{cases}$$

  with $1 \leq i \leq L,\ 1 \leq j \leq K$
- The **output** of **classifier** $\boldsymbol{d_i}$ for **instance** $\boldsymbol{x}$ is a **one-hot vector** $\boldsymbol{d_i(x) = [d_{i1}(x), \ldots, d_{iK}(x)]}$

We have the following **voting schemes** (combiners):

- **Plurality vote**: the class $\hat{y}$ with **more votes** among the classifiers is chosen.

$$\hat{y} = \underset{j=1,\ldots,K}{\arg\max} \sum_{i=1}^{L} d_{ij}(x)$$

  For notational simplicity, we take $\hat{y}$ as the $\hat{y}^{th}$ class. The real **output** would be a **one-hot vector** with 1 at position $\hat{y}$.

- **Thresholded plurality vote**: the ensemble outputs the class $\hat{y}$ if it is the **most voted** and the number of votes **equal or higher than a threshold**.

$$\hat{y} = \begin{cases} k & \text{if } k = \underset{j=1,\ldots,K}{\arg\max} \sum_{i=1}^{L} d_{ij}(x) \text{ and } \sum_{i=1}^{L} d_{ik}(x) \geq \alpha L \\ K+1 & \text{otherwise} \end{cases}$$

  where $0 < \alpha \leq 1$ and class $K+1$ is assigned to instances in which the ensemble lacks confidence or if a tie is reached.

- **Mayority vote**: class $\hat{y}$ needs **at least half of the votes plus one**. A particular case of thresholded plurality vote with:

$$\alpha = \frac{1}{2} + \epsilon, \text{ for an arbitrarily small } \epsilon \text{ such that, } 0 < \epsilon < \frac{1}{L}$$

- **Unanimity**: the vote has to be **unanimous**. A particular case of thresholded plurality vote with $\alpha = 1$.

- **Weighted majority vote**: a generalization of plurality vote in which the **votes** of each classifier are **weighted**.

$$\hat{y} = \underset{j=1,\ldots,K}{\arg\max} \sum_{i=1}^{L} w_i \, d_{ij}(x), \quad \text{and} \quad \sum_{i=1}^{L} w_i = 1, \; w_i \geq 0$$

  where w_i is the **weight** given to classifier d_i, usually because of its **performance**.

2.3 Combining continuously-valued outputs

Given L base classifiers $d_1, \ldots, d_L$, and K classes, the **output** of each learner is a **real-valued vector** of dimension K.

- $\boldsymbol{d_{ij}(x)}$ is a **real number**, expressing the **support** that classifier $\boldsymbol{d_i}$ gives to the hypothesis that **instance** $\boldsymbol{x}$ belongs to **class** $\boldsymbol{j}$ (e.g., a posterior probabability).
- The **output** of **classifier** $\boldsymbol{d_i}$ is still a **vector** $\boldsymbol{[d_{i1}(x), \ldots, d_{iK}(x)]}$

  E.g., the outputs of a ANN before using a threshold to convert them into a binary vector.

These values can be combined to produce a **single output vector** $\boldsymbol{y = [y_1, \ldots, y_K]}$, in which each component $\boldsymbol{y_j}$ is the **overall support** for class j, by **element-wise operations**:

- **Arithmetic mean**: $y_j = \frac{1}{L} \sum_{i=1}^{L} d_{ij}(x)$
- **Minimum**: $y_j = \min_{i=1,\ldots,L} d_{ij}(x)$
- **Maximum**: $y_j = \max_{i=1,\ldots,L} d_{ij}(x)$
- **Median**: y_j is the median value of $d_{1j}(x), d_{2j}(x), \ldots, d_K(x)$
- **Trimmed mean**: arithmetic mean **excluding extreme values**.

  $k\%$ trimmed mean $\Rightarrow k/2\,\%$ highest and $k/2\,\%$ lowest values excluded.

  E.g., The 40% trimmed mean is the arithmetic mean of all but the 20% highest and 20% lowest values.
- **Product**: $y_j = \Pi_{i=1}^{L} d_{ij}(x)$

The class $\hat{y}$ with the **biggest support** is chosen as the **output** of the ensemble. E.g., for the arithmetic mean:

$$\hat{y} = \arg\max_{j=1,\ldots,K} \frac{1}{L} \sum_{i=1}^{L} d_{ij}(x)$$

Same formulation for all the other combination rules: maximum, minimum, median, ..., including their generalization (see below).

E.g., classification of an instance x given by 5 different classifers for 3 classes: (classes with biggest suport in bold)

	Classes		
Learners	d_{i1}	d_{i2}	d_{i3}
d_1	0.1	$\mathbf{0.5}$	0.4
d_2	0	0	$\mathbf{1}$
d_3	$\mathbf{0.4}$	0.3	$\mathbf{0.4}$
d_4	0.2	$\mathbf{0.7}$	0.1
d_5	0.1	$\mathbf{0.8}$	0.2

Outputs of the combiners we have seen for the same instance x:

Combination rule	y_1	y_2	y_3
Mean	0.16	$\mathbf{0.46}$	0.42
Minimum	0	0	$\mathbf{0.1}$
Maximum	0.4	$\mathbf{0.8}$	1
Median	0.1	$\mathbf{0.5}$	0.4
40% trimmed mean	0.13	$\mathbf{0.5}$	0.33
Product	0	0	$\mathbf{0.0032}$

Most of these combiners have an **intuitive meaning**:

- **Arithmetic mean**: **general consensus** of the base classifiers.
- **Median and trimmed mean**: also general consensus, but **preventing** the effect of very **extreme values**.
- **Minimum**: the **most pessimistic** estimate. Learners tell us the chances of class j are at least their lowest estimate.
- **Maximum**: the **most optimistic**. At least one learner believes these are the chances of class j.

We can generalize previous combiners as:

- **Generalized mean**: a generalization of the arithmetic mean governed by a **parameter α**.

$$y_j = \left(\frac{1}{L} \sum_{i=1}^{L} d_{ij}(x)^{\alpha} \right)^{\frac{1}{\alpha}}$$

Changing the value of α we can get **different combiners**.

$$\begin{array}{lcll}
\alpha \to -\infty & \Rightarrow & y_j = \min_{i=1,\ldots,L} d_{ij}(x) & \text{(minimum)} \\
\alpha = -1 & \Rightarrow & y_j = \left(\frac{1}{L}\sum\limits_{i=1}^{L} \frac{1}{d_{ij}(x)}\right)^{-1} & \text{(harmonic mean)} \\
\alpha \to 0 & \Rightarrow & y_j = \left(\Pi_{i=1}^{L} d_{ij}(x)\right)^{1/L} & \text{(geometric mean)} \\
\alpha = 1 & \Rightarrow & y_j = \frac{1}{L}\sum\limits_{i=1}^{L} d_{ij}(x) & \text{(arithmetic mean)} \\
\alpha \to \infty & \Rightarrow & y_j = \max_{i=1,\ldots,L} d_{ij}(x) & \text{(maximum)}
\end{array}$$

The value of $\boldsymbol{\alpha}$ can be conceptualized as the **level of "optimism"** of the combination.

- $\boldsymbol{\alpha \to -\infty} \Rightarrow \alpha$ is at is **most pessimistic**, so the combination is the minimum.
- $\boldsymbol{\alpha \to \infty} \Rightarrow \alpha$ is at is **most optimistic** combination: the maximum.

The value of α can be chosen beforehand, or tuned with respect to the ensemble performance.

- **Weighted average**: the support for class j is calculated as a **linear combination** of the supports d_{ij} of individual base learners, with a set of **weights** $\boldsymbol{w_i}$.

  Weights are usually set so as to express the **quality of the classifiers**. Two possibilites:

  1. One weight **per base classifier**.

     $$y_j = \sum_{i=1}^{L} w_i \, d_{ij}(x)$$

     If $\boldsymbol{w_i = 1/L}, i = 1, \ldots, L$ we go back to the **arithmetic mean**.
  2. One weight **per base classifier and attribute**.

     $$y_j = \sum_{i=1}^{L} w_{ij} \, d_{ij}(x)$$

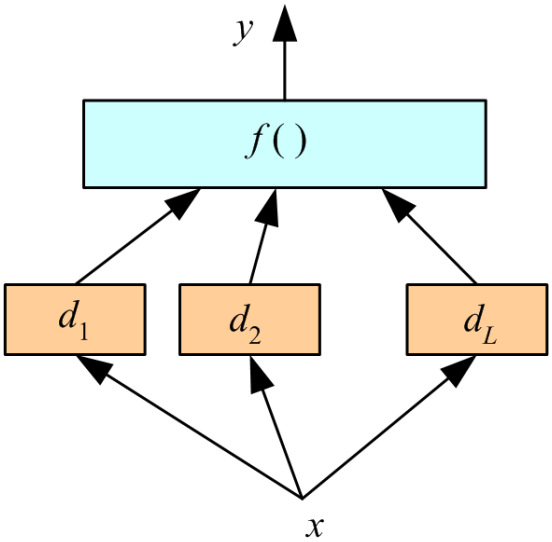
2.4 Stacked generalization

The **problem** with **voting** or combining **continuously-valued outputs**, is that, except when using weights, we just take the outputs of the base learners, without **no differentiation** between them.

- Some base learners may be **more reliable** than others **in general**.
- Some base learners may be **more reliable** for **certain inputs**.

Stacked generalization (usually called **stacking**) tries to solve this issue, learning a **combiner** which receives as its input the outputs of the **base learners**, producing the **prediction** of the ensemble as a result.

- The **combiner** is a **learner** itself, and it has to be trained.
- It is also called **meta learner**, **meta model** or **meta classifier**.



Source: "Introduction to Machine Learning", Ethem Alpaydin, Ch 17

Stacking can be seen as an extension or **generalization of voting**.

- The **combination** of the outputs of base learners does **not** have to be **linear** (e.g., meta model can be a multilayer perceptron),

The combiner learns what the correct output is based on the outputs of the base learners:

- **Goal**: for the combiner to **learn** how the base learners make **mistakes**.
- Stacking is a mean of estimating and **correcting** for the **biases** of base learners.

Stacking is **less used** than other alternatives, like bagging and boosting.

- It is **difficult** to theoretically **conceptualize** it well.
- There are **many** possible **variations**.

It is desirable for the **base learners** to be as **different** as possible, to guarantee **diversity**.

- Usually **base learners** of **different types** are chosen.
- When combining classifiers capable of generating **continuous outputs** (e.g., probabilities) it may be better to take them as the **input** for the **combiner**, rather than classes.

Architecture

- **Two levels** of **learners** are considered.

  Level 0: Base learners

  Level 1: Combiner
- The **predictions** of base learners from level 0 are the **input** of the **combiner**, both in training and prediction.
  - Predictions are concatenated in a single feature vector.
- Most of the work is done by level-0 base learners, with the **meta classifier** acting as an **arbiter** $\Rightarrow$ A **simple classifier** can be used (lineal models, decision trees).

Prediction

1. An **instance** is fed to **level 0** classifiers.
2. Each **level 0** learner produces an **output**.
3. Level-0 predictions are fed to the **meta learner** at **level 1**, which produces the **final prediction**.

Training

- The **meta learner** should **not** be trained on the **same instances** than the **base learners**:

- Because the meta learner is trying to **correct** for the base learner **biases**.
- It can be trained on **held-out data** (not used in the training of the base learners) ot k-fold **cross validation** can be used.

Not everybody follows this convention, training base learners and meta model on the same (or partially overlapping) data.

- **Hold-out**:
  1. We divide the training data in **two disjoint subsets**.
  2. We use **one** to **train** the **base learners**.
  3. We feed **the other** subset to the trained base learners to generate the inputs of the combiner, and **train** the **combiner**.
- **k-fold cross validation**:
  1. Let's use **3 types** of **base learners** and 4-**fold cross validation**:
  2. The initial dataset is divided in **4 disjoint subsets** D_1, D_2, D_3, D_4

Training	D_1	D_2	D_1	D_1
	D_2	D_3	D_3	D_2
	D_3	D_4	D_4	D_4
Test	D_4	D_1	D_2	D_3

  3. For **each type** of **base learner**, **4 classifiers** are trained, one in each of the **4 training sets**: $\{D_1, D_2, D_3\}$, $\{D_2, D_3, D_4\}$, $\{D_1, D_3, D_4\}$, and $\{D_1, D_2, D_4\}$
  4. The **meta learner** is trained on **all instances** of the original dataset:

     If the instance is in D_1, the outputs of the 3 level 0 classifiers trained on $\{D_2, D_3, D_4\}$ are feeded to the meta learner.

     If the instance is in D_2, the outputs of the 3 level 0 classifiers trained on $\{D_1, D_3, D_4\}$ are feeded to the meta learner.

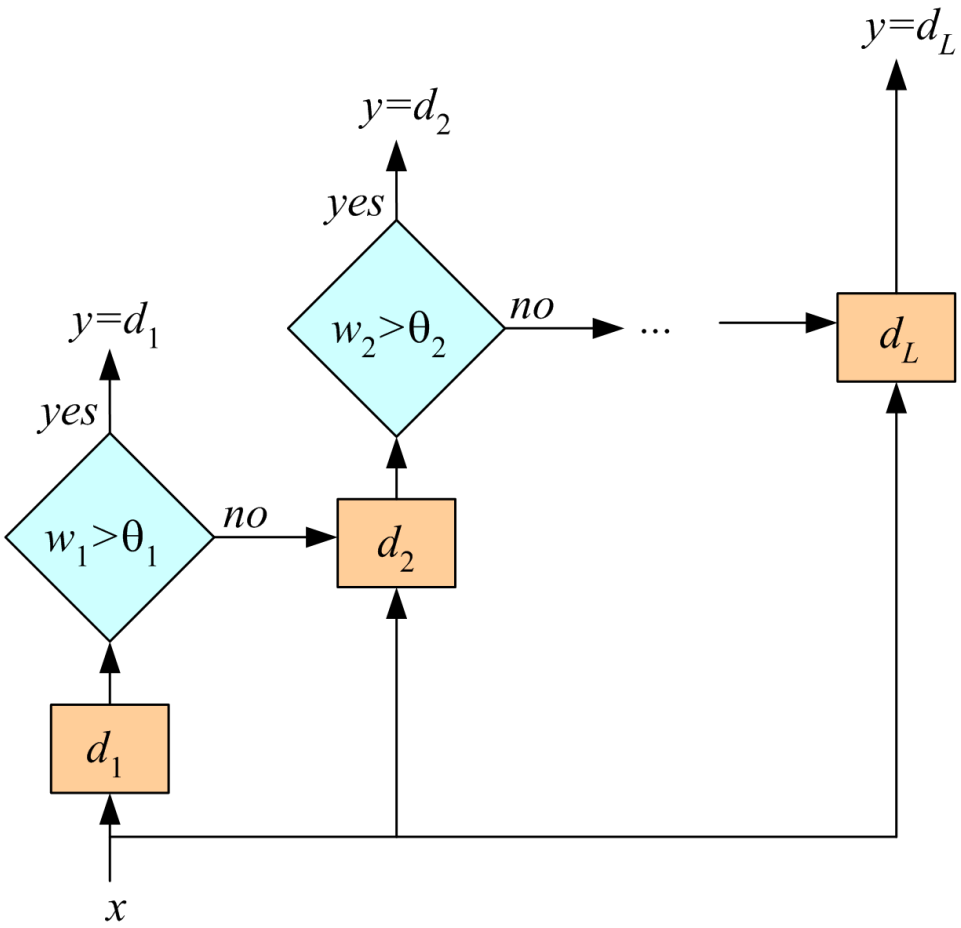
     ...
- This is for training only! To **validate** the ensemble we would need **additional** held-out **data**.

Finally, though the previous discussion is based on classification problems, stacking can also be used for **regression**.

2.5 Cascading

Idea: to have a sequence of L **base classifiers** $d_1, \ldots, d_L$ **sorted** in terms of their space or time **complexity**:

- d_{i+1} is **costlier** than d_i, with $1 \leq i < L$.
- The **prediction** of d_i is the output of the ensemble if all preceding classifiers d_j, $1 \leq j < i$ are **not confident** enough in their predictions.
  - Each **learner** d_i gives a **confidence** w_i associated to its prediction.
  - Learner d_i is confident enough of its output if $w_i > \theta_i$, with θ_i the **confidence threshold** of learner d_i, and $1/K < \theta_i \leq \theta_{i+1} < 1$.
  - The **confidence** function is set to the **posterior** probability of the **predicted class** (the one with largest posterior): $w_i = \max_j d_{ij}(x)$.



Source: "Introduction to Machine Learning", Ethem Alpaydin, Ch 17

Training

1. Given a **training dataset**, we train d_1.
2. We make predictions for all instances in a separate **validation set**, and find those in which d_1 is **not confident**.
   - They may include both misclassified and correctly classifed instances.
   - We use them to train the **next classifier** d_2.
3. Repeat until we have $\boldsymbol{L}$ **base learners**.

Goal: for an early **simple** classifier to handle the **majority of instances**.

- More **complex** classifiers are used only for a **small percentage** of these instances.
- Thereby the increase in overall **complexity** is **minimized**.

There may be a small number of **instances** for which predictions are **not confident** enough in all base learners.

- They are **stored** as they are and treated by a **nonparametric classifer**, such as k-NN.

The inductive process of cascading is that **classes** can be **explained** by a small number of **"rules"**, applied in **order** of increasing **complexity**, with a small number of **"exceptions"** not covered by the rules.

- **Rules** are implemented by **simple base classifiers** (e.g., perceptrons of increasing complexity).
- Classifiers learn rules applicable over the **whole input space**, to explain instances as **cheaply** as possible.
- **Exceptions** are handled by a nonparametric model.
- Cascading is a **"middle ground"** between extremes of **parametric** (a single rule) and **nonparametric** classification (no rules).

2.6 Bagging

Bagging (**B**oostrap **Agg**regat**ing**): **voting** method in which base learners are made **diverse** by training them over slightly **different training sets**.

- L slightly different samples are generated from a dataset by **bootstrap**, and each one is used to train base **learners**, usually of the **same type**.
- For **discrete outputs**, predictions are made by **voting**.
- For **continuous outputs**, prediction is the **average** or **median**.

Bootstrap: given a training set of size n, n instances are **randomly** drawn from the training set **with replacement**. That makes it possible:

- for some **instances** to be drawn **more than once**, and
- for other **instances** to not be drawn **at all**.

Samples obtained from boostrap on the same dataset are **similar** because they are all drawn from the **same original sample**, but they are also slightly **different** due to **chance**.

- The **probability** for an instance of being selected as part of a bootstrap sample is $1 - \left(1 - \frac{1}{n}\right)^n$, about $\mathbf{0.632}$ for large n values.
- That means it is expected for each bootstrap sample to contain about 63.2% of the instances in the original training dataset.

When doing bootstrap on very large training sets, we may want to **generate smaller** sets of size $n' < n$.

- Otherwise bootstrap replicates may be **too similar**, reducing diversity and making bagging not useful.

How high should $\boldsymbol{L}$ be?

- Initial experiments with decision trees (Breitman) suggested a range of **25-50 base learners** as sufficient to get the ensemble error to level off.
- Experiments with other learners has shown that range to be a good rule of thumb.

Training

1. Let n be the number of **instances** in the training set.
2. For $i = 1, \ldots, L$:
   (a) A bootstrap **sample** S_i (size n) is taken from the training dataset.
   (b) A classifier d_i is **trained** on that sample and added to the ensemble.

Prediction

1. For $i = 1, \ldots, L$:
   (a) The **prediction** of each classifier $d_i(x)$ is computed.
   (b) The most frequent prediction is chosen by **plurality vote**.

Bagging tends to produce **better** results with **unstable** learning algorithms.

- **Small changes** in the training **data** tend to create **big differences** between the trained **learners** (learning algorithm has **high variance**). E.g., decision trees or multilayer perceptrons.
- By using **different samples** on the same type of learners, **diversity** among base learners is ensured.
- On the other hand, the **variance** of a bagging ensemble is **lower** than the variance of the learning algorithms used in the **base learners**.
  - **voting** or **averaging** reduce **sensitivity** to individual data points.
- This effect is less pronounced with **stable learners** $\Rightarrow$ **lower or no improvement** when using bagging on them.

Types of outputs from the base learners and how to combine them:

- **Categorical output** (class) $\Rightarrow$ **plurality voting**.
- **Class probabilities** (posterior probabilities). Two options:
  1. **Plurality voting**: class with the highest probability in each learner.
  2. **Average** the probabilites of each class obtained by each base classifier an choose the class with **highest** average.

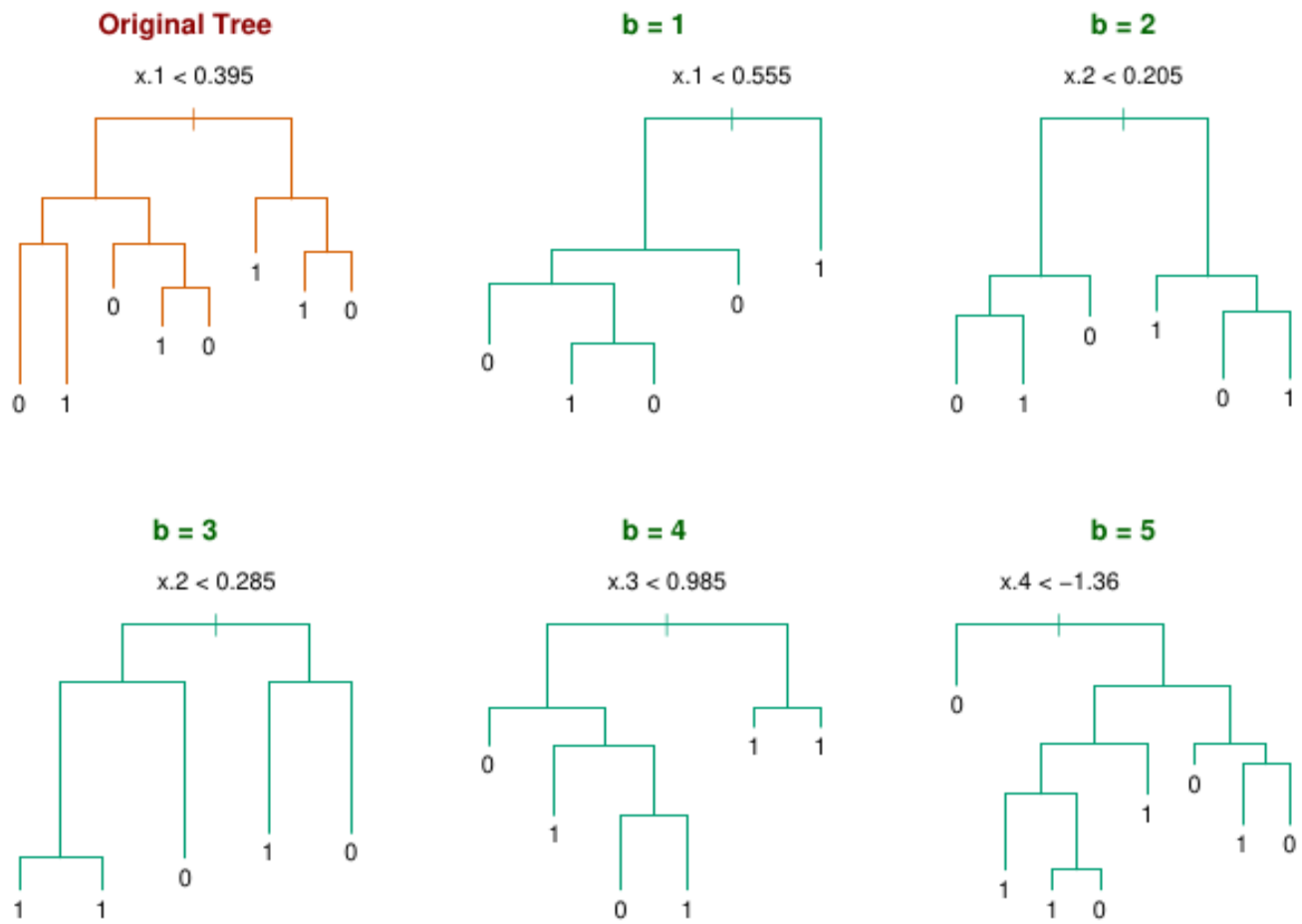
  Both produce **similar error rates**.
- **Numeric predictions** (regression) $\Rightarrow$ **average** or **median**.

Let's see a toy example of how bagging works in practice, taken from the book "Elements of statistical Learning" by Hastie, Tibshirani & Friedman (Example 8.7.1)

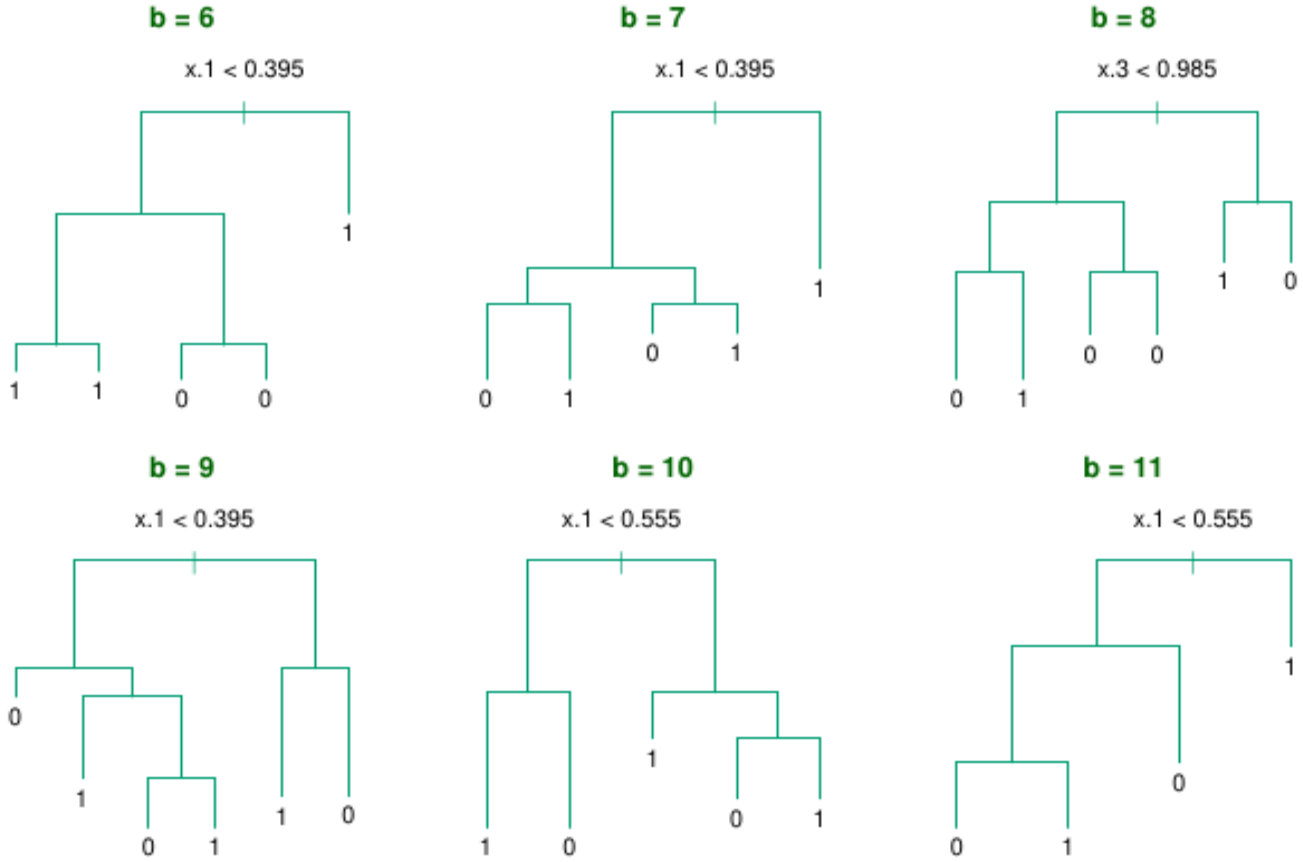
The authors generate a synthetic training set of size $n = 30$, for binary classification, with 5 features.

- Each feature has a standard Gaussian distribution with pairwise correlation 0.95 (very high).
- Class is generated according to $P(Y = 1 \mid x_1 \leq 0.5) = 0.2$ and $P(Y = 1 \mid x_1 > 0.5) = 0.8$.

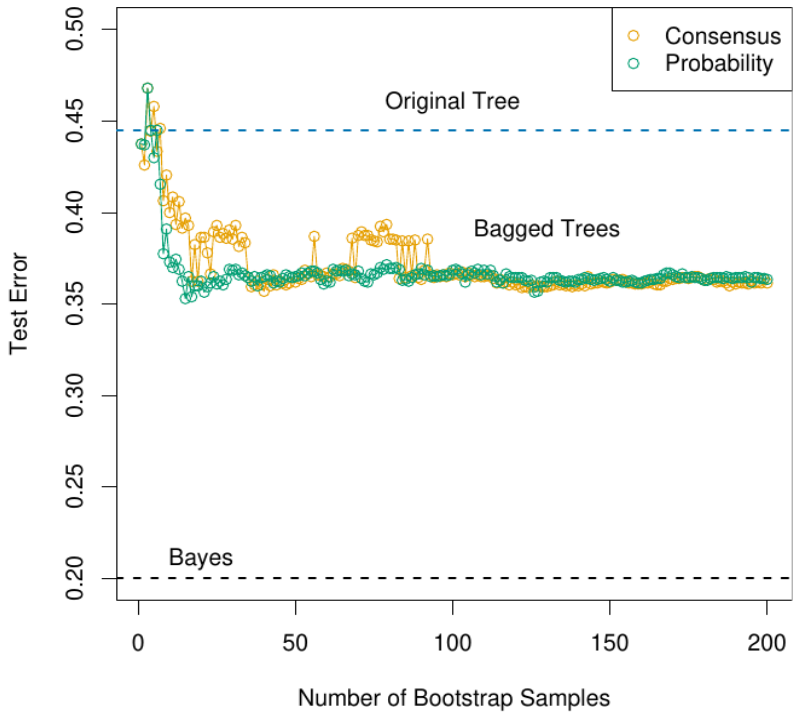
A validation dataset from the same population and size 2000 is also generated. The authors take 200 bootstrap samples and use them to train the same number of classification trees, without pruning.



- No clear difference with respect to which feature to split.
- Small changes in the training set produce very different trees.



They also draw error curves relative to bootstrap samples (orange corresponds to hard voting and green to average probabilities).



- No performance improvement with more than 30-50 trees.
- It looks like trees are more correlated than we would like.

2.7 Randomization

Bagging introduces **randomness** in the **inputs** of the base learners.

There are **other ways** to create diversity, by introducing **randomness** into the **base learners** themselves.

- Any learning algorithm is subsceptible of **randomization** at some stage.

  E.g., a k-NN determines the class of an instance by computing its distance from already classified instances in the feature space.
  - We can randomly choose a subset of all features to compute distances.
- We get **more diversity**, at the expense of **worse use of data**, even to the point where the performance of the base learners can be negatively affected.
  - We need to strike a **balance** between **diversity** and **performance**.

Randomization vs **bagging**:

- **Randomization** is **more costly** than bagging because the base learning algorithm has to be modified.
- On the other hand, it performs **better** than bagging when applied to **estable** learning algorithms.

Random Forests

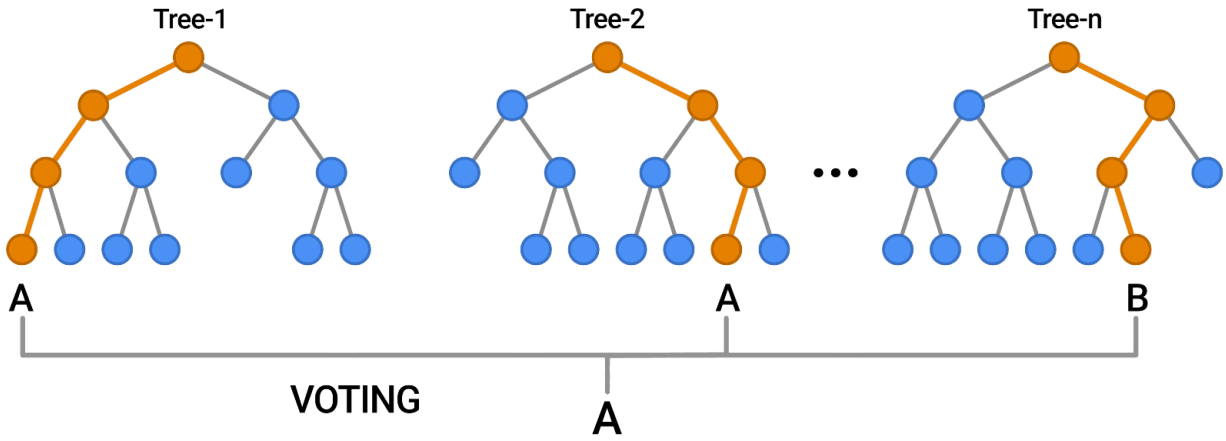
Random forests represent a general class a ensemble methods using **decision trees** as base classifiers, introducing **randomization**:

- Each tree is grown with respect to a **random vector** $\vec{\theta}$, where all $\theta_1, \ldots, \theta_L$ are independent and identically distributed.
- **Randomness** can be introduced by:
  1. **Sampling** from the **data set**.
  2. **Sampling** from the **feature set**.

     When choosing a **feature** to **split** a node, instead of the best:
     (a) Choose one **randomly**.
     (b) Choose the **best feature** from a **randomly selected subset**.
     (c) **Randomly** choose one among the **best**.

3. Varying randomly **other parameters** of the tree.

- Predictions are made by **plurality vote** among the base tree learners (or **averaging** for **regression** problems).



Source: blog.tensorflow.org

Originally, random forests (Breitman) were introduced as a version of **bagging** using **decision trees** as the base classifier:

- A different **bootstrap sample** is chosen for each **base learner** from the training data.
- Two variants:

  Forest-RI: when splitting a node in training.

  1. Given M **features**, pick F of them **randomly**.
  2. Choose the **best** of the F dimensions to do the split.

  – A **new feature subset** of size F is chosen **in each node**.
  – F is a **parameter** of the algorithm:
    * **Small value**: **diverse** base trees, but also **worse** performing.
      $F = 1 \Rightarrow$ random selection $\Rightarrow$ maximum diversity
    * **Large value**: **correlated** trees, but **better** performing ones.
      $F = M \Rightarrow$ best selection $\Rightarrow$ always the same tree
  – **Recomendation** from Breitmen:
    * CART tree (Gini index for impurity reduction).
    * No pruning.
    * $F = \lfloor \log_2 M + 1 \rfloor$, where $\lfloor\ \rfloor$ is the floor function.

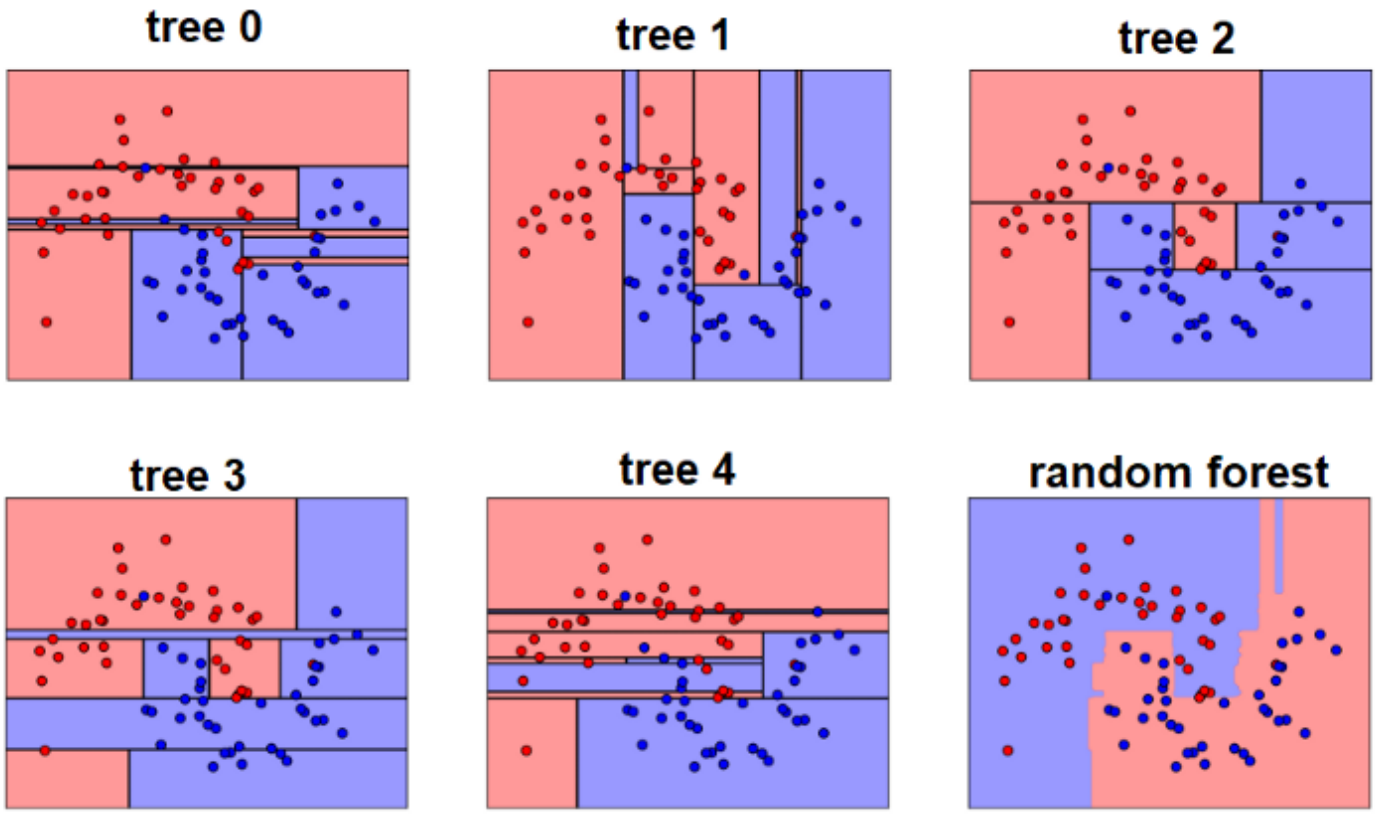
Forest-RC: when the number of features M is small.

- Increase the feature space with **linear combinations of features**.
- At each node:
  1. F **randomly combined** features are **created**.
  2. Each one one of these features is a **linear combination** of:
     * L randomly selected **input features**.
     * **Coefficients** generated from a uniform distribution in $[-1, 1]$.
  3. The **best** of those F combined features is selected to split the node.

Random forest ensembles can be seen as a way to reduce **overfitting** inherent in decision trees by building (not so) slightly different decision trees and averaging their results.

E.g., in this tutorial a random forest of 5 individual trees is built, on a dataset developed by the make_moons generator of scikit-learn.

The decision boundaries of each classifer are plotted in the following figure.



While the performances of the individual trees and the ensemble seem to be similar, the latter presents less overfitting through a more "continuous" decision boundary.

2.8 Boosting

Boosting is a family of ensemble methods designed to obtain **strong** learners from **weak ones**.

- A **weak learner** is a model with **error** lower than the error made by **random guessing** $\Rightarrow$ **cheap** and **easy** to obtain.
- A **strong learner** is a model with an **arbitrarily small** error rate $\Rightarrow$ **difficult** to build.

Boosting works by **incrementally** building an ensemble of weak learners.

a) A **new learner** is trained in **each iteration**.

b) The training set of each iteration is **selectively sampled**.
   - At iteration $i = 1$ all instances in the training set have the **same probability** of being selected.
   - In subsequent iterations $1 < i \leq L$ **misclassified** instances in the previous iteration $i - 1$ have their **probabilities** of being selected **increased**.

   Each new learner is preferently trained on the mistakes of the previous ones.

Goal: to actively generate **diverse** models that **complement** each other.

- Each weak learner **especializes** in a part of the training data, determined by the errors of previous learners.
- In contrast with **bagging**, in which diversity depends on the **randomness** of sampling and the **instability** of the base learners.

The most consequential of the boosting algorithms is AdaBoost.

AdaBoost (Adaptive Boosting)

AdaBoost incrementally trains weak clasifiers d_i, with $1 \leq i < L$, with error rates lower than 0.5 (the error rate of a random guesser on a binary classification problem).

Since each weak learner d_i has an specific focus on previously misclassified instances, it should be **simple** and **not very accurate**,

- otherwise it would risk **overfitting** over a few misclassified **outliers** present many times in the training set.

Training (multiclass classification): Given a labeled training dataset $(x_1, y_1), \ldots, (x_n, y_n)$ of n instances, an ensemble size L, and a base learning algorithm.

1. Initialize **weights** $w_1 = [w_1^1, \ldots, w_1^n],\ w_1^j \in [0,1],\ \sum_{j=1}^{n} w_1^j = 1$
   (Usually $w_1^j = 1/n,\ j = 1, \ldots, n$)
2. For $i = 1, \ldots, L$
   (a) **Sample** S_i from the training dataset with **distribution** w_i.
   (b) Build a **classifer** d_i using S_i as a training set.
   (c) **Predict** with d_i on the original dataset and calculate the **weighted ensemble error** at step i by:
   $$\varepsilon_i = \sum_{j=1}^{n} w_i^j \times \begin{cases} 0 & \text{if } d_i(x_j) = y_j \\ 1 & \text{otherwise} \end{cases}$$
   (d) if $\varepsilon_i = 0$ or $\varepsilon_i \geq 0.5$, set **weights** w_i^j to $1/n$ and go back to (a). Else calculate the **importance** α_i of classifier d_i:
   $$\alpha_i = \frac{1}{2}\ \ln\left(\frac{1-\varepsilon_i}{\varepsilon_i}\right) \quad \text{(the lower } \varepsilon_i \text{ the higher } \alpha_i \text{ is)}$$
   and **update individual weights** w_i^j with α_i:
   $$w_{i+1}^j = \frac{w_i^j}{Z_i} \times \begin{cases} e^{-\alpha_i} & \text{if } d_i(x_j) = y_j \\ e^{\alpha_i} & \text{otherwise} \end{cases} \quad j = 1, \ldots, n$$
   with **normalization** Z_i term to ensure $\sum_{j=1}^{n} w_{i+1}^j = 1$.
3. **Return** $\mathcal{D} = \{d_1. \ldots, d_L\}$ and $\alpha_1, \ldots, \alpha_L$.

Effect of weight updates:

$$\textbf{if } \boldsymbol{d_i(x_j) = y_j}\textbf{:}\ \ e^{-\alpha_i} < 1 \Rightarrow w_i^j\, e^{-\alpha_i} < w_i^j \Rightarrow w_{i+1}^j < w_i^j$$

$$\textbf{otherwise:}\ \ e^{\alpha_i} > 1 \Rightarrow\ \ w_i^j\, e^{\alpha_i} > w_i^j \Rightarrow w_{i+1}^j > w_i^j$$

There are two basic variants of AdaBoost:

- With **resampling**: the one shown in the **previous pseudocode**.
- With **reweighting**: it is assumed that the **base classifiers** can directly **use** w_j^i as **instance weights**, training with the whole dataset.
  - **No sampling** S_i is necessary, and the algorithm is **deterministic**.

  There is no strong evidence favoring one of the versions over the other.

Prediction: Given $\mathcal{D} = \{d_1 \ldots, d_L\}$, $\alpha_1, \ldots, \alpha_L$, and a new instance x.

1. **Classify** x by **all learners** $d_1 \ldots, d_L$.

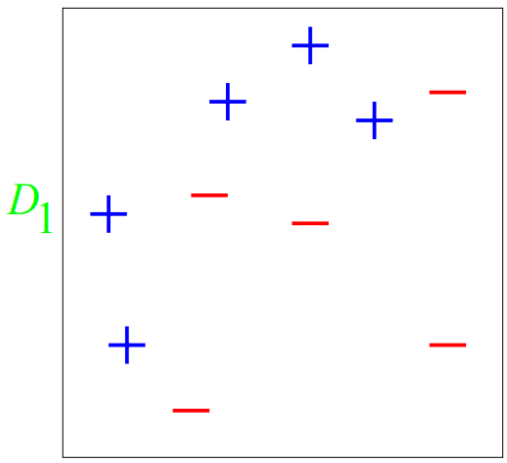
   Outputs are **one-hot vectors** $d_i(x) = [d_{i1}(x), \ldots, d_{iK}(x)]$.
2. We compute the **support** y_j of each class j as:

$$y_j = \sum_{i=1}^{L} \alpha_i \, d_{ij}(x) = \sum_{d_{ij}(x)=1} \alpha_i$$

3. The class with the **maximum support** is chosen as the label for x.

To see how it all works, we will use a small example adapted from "A Tutorial on Boosting" by Freund & Schapire.

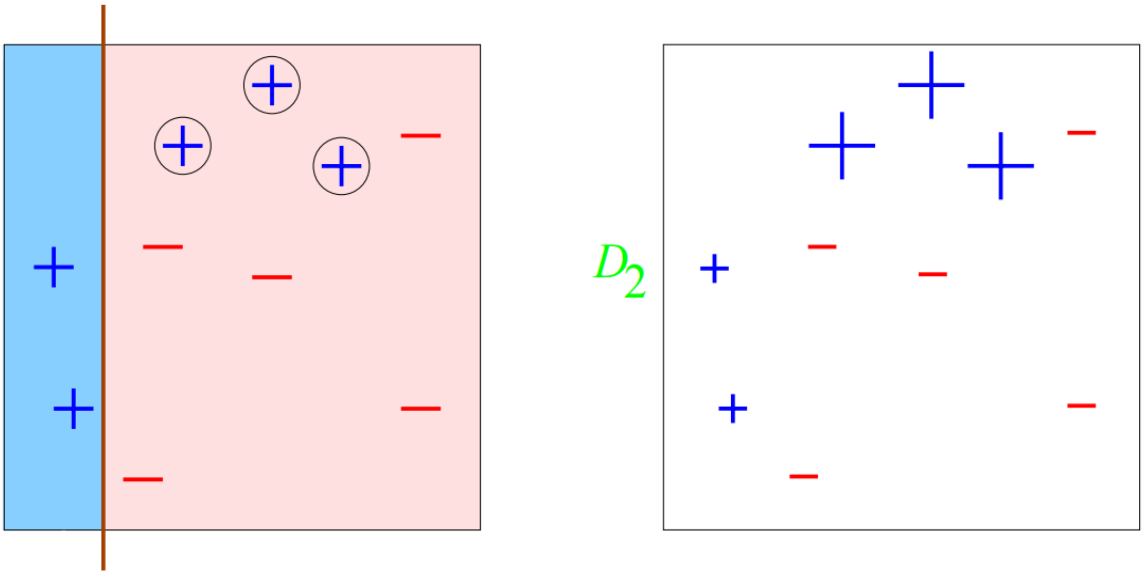
- We have a **binary classification** problem, with **classes** $\{-1, +1\}$, just two features and the following dataset:



- **Predictions** are computed in a slightly different manner than the one previously seen for multiclass problems.

$$y = sign\left(\sum_{i=1}^{L} \alpha_i \, d_i(x)\right)$$

- **Iteration 1**: First weak classifer with error $\varepsilon_1 = 0.30$ and $\alpha_1 = 0.42$ is trained and applied to the dataset.

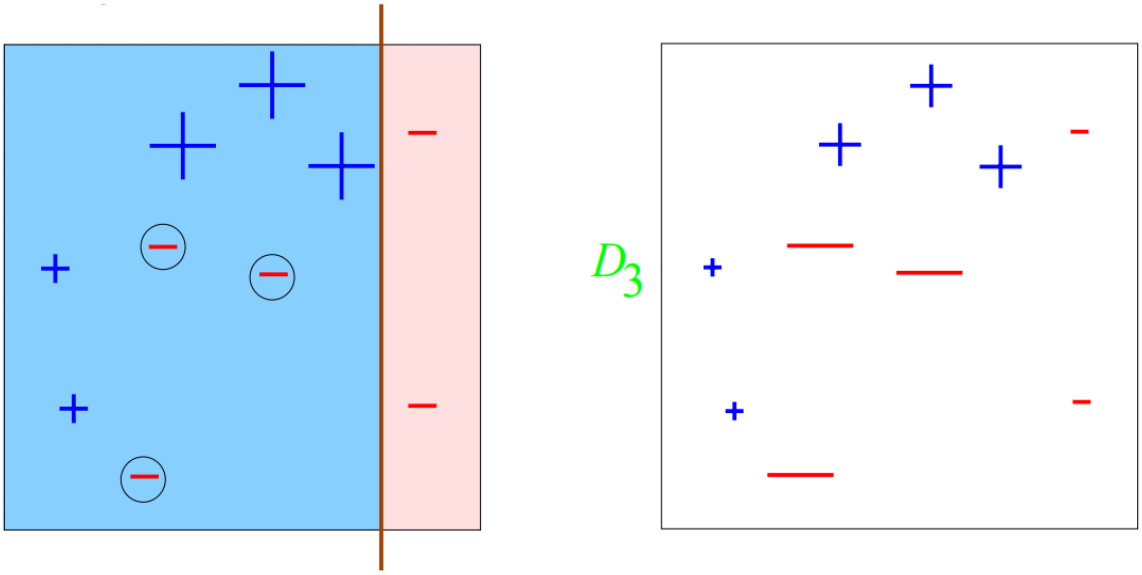


On the **left**, the result of the classification:

  - **Vertical brown line** is the **decision boundary**.
  - **Positive** (resp. negative) predictions in the **blue** (resp. red) **region**.
  - **Three positive examples** have been **misclassified**.

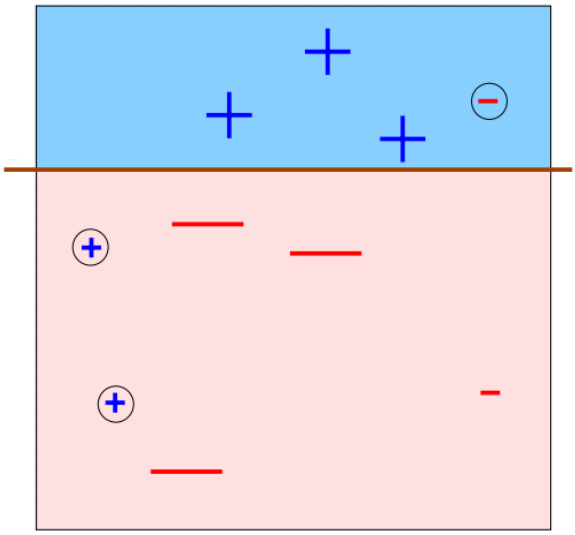
  On the **right**, the dataset reflecting the **weights' update**. Misclassified examples have larger weights.

- **Iteration 2**: Second weak classifer with error $\varepsilon_2 = 0.21$ and $\alpha_2 = 0.65$ is trained and applied to the dataset.



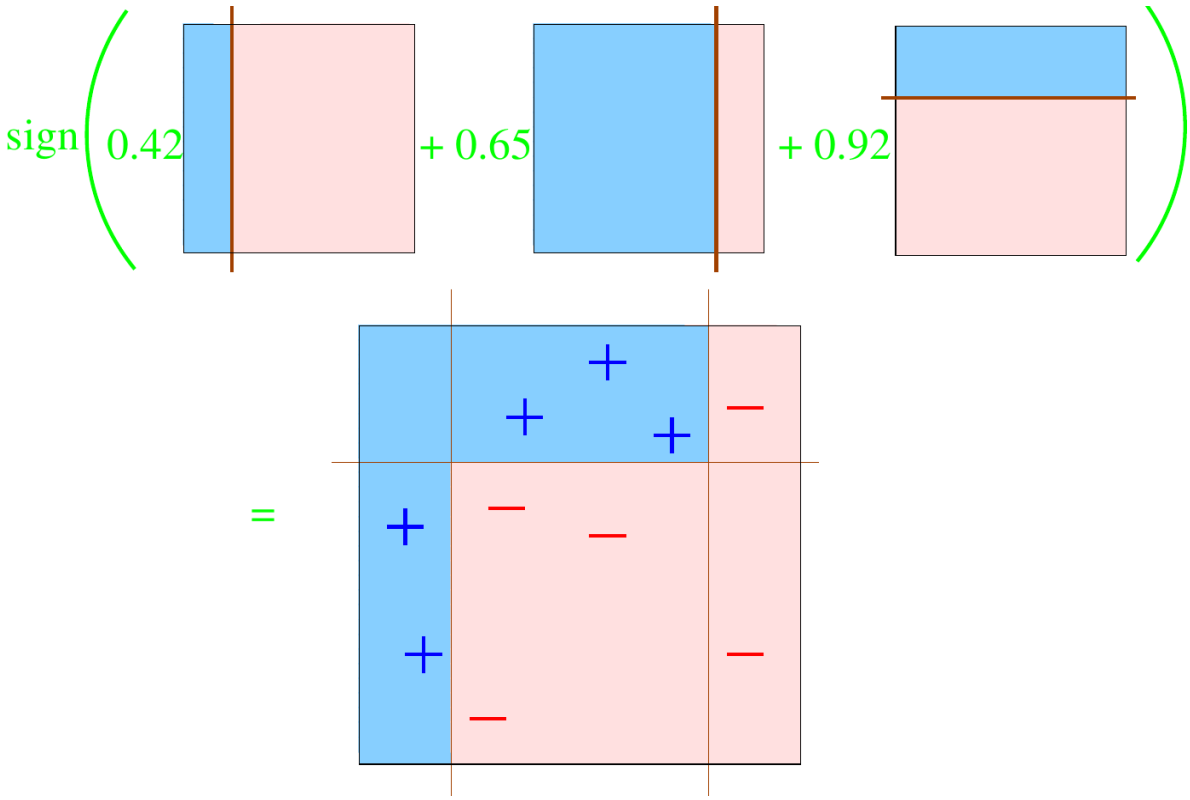
Now **three negative instances** have been **misclassified** (left) so their relative weights are increased (right)

- **Iteration 3**: Third weak learner with error $\varepsilon_3 = 0.14$ and $\alpha_3 = 0.92$ is trained and applied to the dataset.



This time **two positive** and **one negative** examples have been **misclassified**. This is the last weak learner.

- Finally, the **predictions** made by the ensemble on the training set.



- The learners are **complementary** (different instances are misclassified each time) $\Rightarrow$ **good diversity**.
- AdaBoost turns three weak learners into a strong one (at least for this toy example).

AdaBoost: pros and cons

AdaBoost have some **advantages**...

- **Simple** conceptually, and in its implementation.
- A **single parameter** to tune, L, once a type of weak learner is chosen.
- **Speed**: weak learners tend to be chosen by its simplicity, so they tend to be fast.
- Proved to be **effective with weak learners** (in the mathematical sense of the word "proved").
  - As long as the **weak learner** is better than **random guessing**.
  - Almost **any classifier** can be used as weak learner.
- Tends to be **good with outliers**, since they tend to be hard to classify, so their probabilities are going to be increased with each iteration.

... with a couple of **weak points**:

- **Performance** is dependent on **data** and the **weak learner** chosen.
  - AdaBoost have been found to be susceptible to **noisy datasets**.
- AdaBoost **can fail** if:
  - Weak learners are **too complex** $\Rightarrow$ **overfitting**
  - Weak learners are **too weak** $\Rightarrow$ **underfitting** (or, in some circumstances, overfitting)

Gradient Boosting

It is a generalization of AdaBoost combining **boosting** and **gradient descent**.

Similarities to AdaBoost:

- **Sequence** of **weak learners**, whose outputs are added to get the prediction of the ensemble.
- The **output** of the each learner **constrains** how the next weak learner is going to be trained.

Differences with AdaBoost:

- In Gradient Descent all weak learners are trained with the **same dataset**. **No weights** w_j^i are computed, so **no sampling**.
- Gradient descent is conceptualized as a the process of **minimization** of a **loss function** by **iterative gradient descent**. In each iteration, a model F_{i+1} is build from the previous F_i adding a weak learner d_{i+1}.

$$\begin{aligned}
&F_0(x) = d_0(x)\\
&F_1(x) = F_0(x) + d_1(x)\\
&\dots\\
&F_i(x) = F_{i-1}(x) + d_i(x) = \sum_{k=0}^{i} d_k(x)
\end{aligned}$$

  where (x, y) is an instance, and $d_0(x)$ is an initial estimate, not a learner.
- In order to **train** a weak learner d_i, Gradient Descent uses the **residuals** of the previous weak learners as the **real output**, instead of y.

$$r_{i-1}(x) = y - F_{i-1}(x) = y - \sum_{k=0}^{i-1} d_k(x)$$

  the first estimate $d_0(x)$ makes possible to train the first weak learner d_1 on the residuals of d_0.
  - In **binary classification** or regression, r is a single **substraction**, while in **multiclass classification** is a element-wise **vector substraction** between y and $d_i(x)$.
  - This way, weak learner d_i is trying to **correct** the "mistakes" of **previous learners**, and $F_i(x)$ is closer to y than $F_{i-1}(x)$.
- Mathematically speaking, **fitting to the residuals is equivalent to fitting to the negative gradients of a specific loss function for the ensemble**.
  - Each new learner works as an iteration in the optimization of the loss function through **functional gradient descent**.
- The process of producing new weak learners is repeated until the loss function is close enough to zero or until L weak learners are trained.

To make a **prediction** on a instance x, all weak learners are fed with the test data and their outputs added to generate the final prediction.

- In **multiclass classification**, $d_i(x) = [d_{i1}(x), \ldots, d_{iK}(x)]$, and $d_{ij}(x)$ are **real values**, instead of binary as in AdaBoost.
- **Support** y_j for **each class** j is computed as $y_j = \sum_{i=0}^{L} d_{ij}(x)$, and the class with the maximum support is chosen as the label for x.

Gradient boosting is tipically used with **decision trees** as weak learners, but any weak learner will work. **XGBoost** (from **E**xtreme **G**radient **B**oosting) is a modern and popular implementation of Gradient Boosting with decision trees as weak learners, well adapted for large datasets.

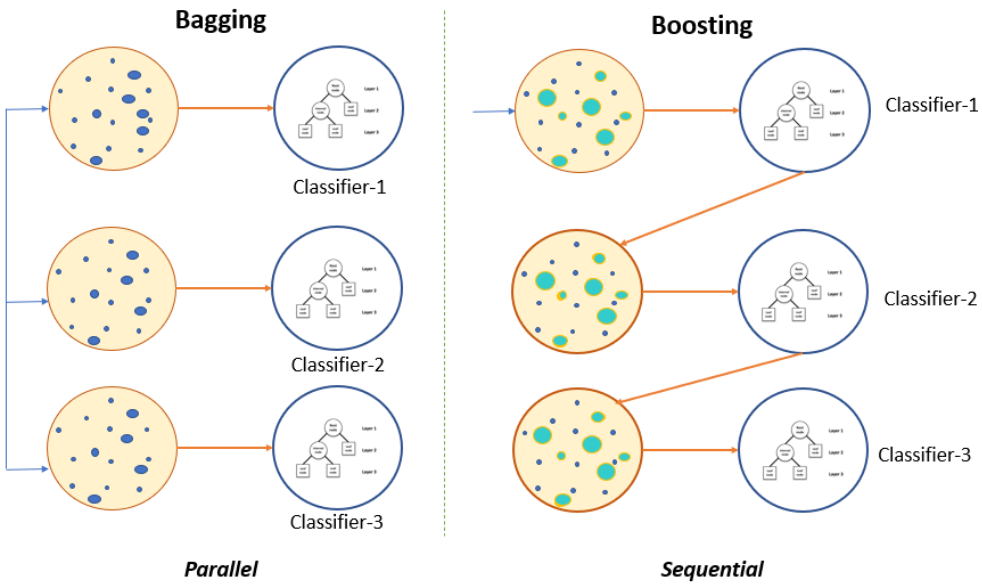
Bagging vs Boosting

Bagging and boosting are two of the most popular techniques to develop ensembles. Lets see its **commonalities**:

- Both use base learners of the **same type**.
- **Majority** (or, more accurately, plurality **vote**) is used to combine the outputs of the base classifiers.
- Base classifiers are trained on **samples** of the **same dataset**.

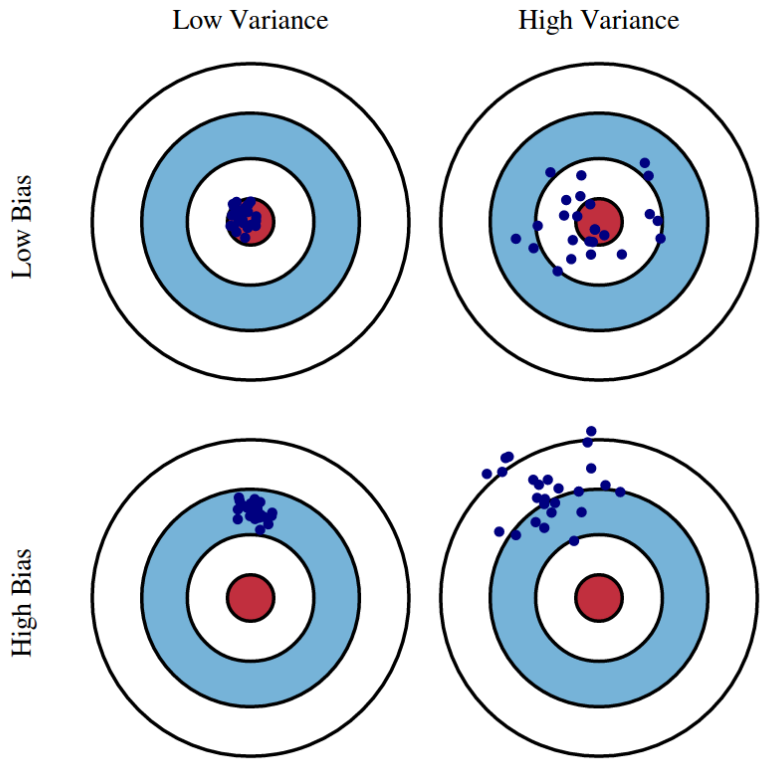
And **differences**:

- In **bagging**, learners can be trained in **parallel**, while **boosting** requires **sequential** training.



Source: https://www.pluralsight.com

- **Base learners** are **independent** in **bagging**. In **boosting**, each new weak learner is supposed to become an **expert** in previously **misclassified instances**.
- **Sampling** from the training set is **random** in **bagging**, while based on the **probabilities** of the instances in **boosting** (and no sampling is done in AdaBoost with reweighting or gradient boosting).
- In **bagging**, each model has the **same weight** in the calculation of the final prediction, while the point of **boosting** is that different learners have **specific weights** based on their error rates on the training set.
- **Bagging** tends to **reduce** the **overfitting** present in its base learners, while **boosting** is more **prone** to it, if its weak learners are too **complex** for the task at hand.
- **Bagging** works better with base learners with **low bias** and **high variance**, by building an ensemble that reduces the latter while keeping the former. **Boosting** works better with models with **high bias** and **low variance**, getting better results by reducing the former.
  - Though it has been shown that **AdaBoost** reduces **bias** in **early** iterations and **variance** in **latter** ones.



Source: "Understanding the Bias-Variance Tradeoff", by Scott Fortmann-Roe